# Fitness API Security Assessment & Hardening Framework

Week 2 of 12 — Introduction to OWASP API Security & Reconnaissance [Foundation]

Submission Report

| | |
|---|---|
| **Author** | chocholucky33@gmail.com |
| **Date** | 2026-10-21 |
| **Program** | JPR Solutions · Internship Project |
| **Target** | Local practice Flask API (127.0.0.1:5000) |
| **Scope** | Reconnaissance & mapping only — no exploitation |

# 1. Engagement Context & Scope

This report documents Week 2 of the Fitness API Security Assessment & Hardening Framework internship project. The objective this week is foundational: to study the OWASP API Security Top 10 and perform initial reconnaissance against the target Flask API, producing an endpoint map and a summary of the OWASP risks.

**Note on the target:** the course materials did not supply a live target API (no URL, application, or dataset). To carry out genuine reconnaissance, a minimal stand-in Fitness Flask API was built and run locally on `127.0.0.1:5000`, and all recon was performed against that authorized, locally-owned instance. The methodology, scripts, and findings transfer directly to the real target once provided.

**Scope & ethics:** this week covers reconnaissance and attack-surface mapping only. No authorization bypass or exploitation was performed. The OWASP "candidate" notes record where testing should focus in later weeks.

# 2. OWASP API Security Top 10 (2023) — Summary of Understanding

Each risk below is described with a fitness-API example, how it is tested during recon, and the core mitigation that drives the later hardening phase.

| Risk | What it is | How to test | Mitigation |
|---|---|---|---|
| API1 — Broken Object Level Authorization (BOLA) | Object IDs are exposed but ownership is not checked, so changing an ID reaches another user's data. The most common and damaging API flaw. | Swap an object ID (e.g. `/users/42/profile`) for one owned by another user and see if data returns. | Enforce an ownership check on every object access; prefer unguessable IDs as defence-in-depth. |
| API2 — Broken Authentication | Weak/missing auth: no login rate limiting, guessable or unverified tokens, credentials in URLs. | Attempt login without credentials, inspect/decode tokens, test tokens after logout/expiry. | Strong passwords, rate limiting + lockout, short-lived signed tokens, verify JWT alg/signature. |
| API3 — Broken Object Property Level Authorization | Returns more fields than allowed (data exposure) or accepts fields the user shouldn't set (mass assignment). | Inspect responses for sensitive fields; send extra fields in writes and check persistence. | Allow-list fields for both serialization and request binding; never bind bodies onto ORM models. |
| API4 — Unrestricted Resource Consumption | No limits on requests, payload size, or expensive operations — enables DoS and cost spikes. | Send large `limit`/page values and oversized payloads (in scope). | Rate limiting, pagination caps, payload-size limits, timeouts, quotas. |
| API5 — Broken Function Level Authorization (BFLA) | A normal user can invoke admin functions or privileged methods. | As a low-privilege user, call admin routes and PUT/DELETE methods. | Deny-by-default role checks on every endpoint, enforced server-side. |
| API6 — Unrestricted Access to Sensitive Business Flows | A legitimate flow (sign-up, rewards) can be automated and abused at scale. | Identify high-value flows and assess bot-abuse feasibility. | Behaviour/device detection, CAPTCHA where appropriate, per-flow rate limiting. |
| API7 — Server Side Request Forgery (SSRF) | The API fetches a user-supplied URL, letting an attacker reach internal services/metadata. | Supply internal/loopback/metadata URLs where the API fetches remote resources. | Allow-list schemes/hosts, block private ranges and metadata IPs, disable redirects. |
| API8 — Security Misconfiguration | Debug mode, verbose errors, missing security headers, permissive CORS, default creds. | Trigger errors for stack traces, check headers, test CORS with a foreign Origin. | Hardened per-environment config, disable debug, security headers, least-privilege CORS. |
| API9 — Improper Inventory Management | Forgotten old versions and undocumented 'shadow' endpoints stay exposed and unpatched. | Enumerate versions (v1/v2/beta), look for /swagger or /openapi.json, diff environments. | Maintain a full API inventory, retire old versions, document every endpoint. |
| API10 — Unsafe Consumption of APIs | Blindly trusting data returned by third-party APIs the app itself calls. | Review outbound integrations and how their responses are validated. | Validate/sanitize third-party data, use TLS, treat upstream data as untrusted. |

**Key takeaways:** authorization flaws (BOLA/BFLA) dominate API risk and receive most test effort; recon must yield a complete endpoint and parameter inventory before testing (ties to API9); hardening must be deny-by-default, schema-validated, rate-limited and environment-specific.

# 3. API Endpoint Map — Reconnaissance Results

**Target:** http://127.0.0.1:5000 · **Tools:** curl, custom Python recon script (recon.py), manual observation.

**Method:** (1) start the target API; (2) enumerate candidate paths/methods via status-code probing; (3) confirm each live endpoint with curl, recording method, parameters and a response sample; (4) note the OWASP risk each endpoint is a candidate for.

## 3.1 Endpoint Inventory

| # | Method | Path | Parameters | Auth | Status | Response sample | OWASP |
|---|---|---|---|---|---|---|---|
| 1 | GET | /api/health | — | none | 200 | {"service":"fitness-api","status":"ok","version":"v1"} | API9 |
| 2 | POST | /api/login | body: username | none | 200 | {"message":"login endpoint","received_user":"test"} | API2 |
| 3 | GET | /api/users/<id>/profile | path: user_id | none | 200 | {"id":1,"name":"Alice","plan":"pro"} | API1 (BOLA) |
| 4 | GET | /api/users/<id>/workouts | path: user_id; query: limit | none | 200 | [{"id":10,"minutes":30,"type":"run"}] | API1, API4 |
| 5 | POST | /api/users/<id>/workouts | path: user_id; JSON body | none | 201 | {"echo":{},"message":"created"} | API3 |
| 6 | GET | /api/admin/stats | — | none | 200 | {"total_users":2,"total_workouts":2} | API5 (BFLA) |

Probed but absent (useful negatives): `GET /swagger` → 404, `GET /openapi.json` → 404 (no machine-readable spec; manual mapping required).

## 3.2 Parameter Notes

• **user_id** (path integer, endpoints 3–5): directly indexes an object → primary BOLA target; sequential integers are trivially enumerable.
• **limit** (query integer, endpoint 4): controls result size → API4 if uncapped.
• **username / request body** (endpoints 2, 5): user-controlled input echoed/processed → check API3 and input validation.

## 3.3 Attack-Surface Summary

• 6 live endpoints across two resource groups (users, admin) plus a health route.
• No authentication observed on any route, including `/api/admin/stats` → high-priority area for testing (API2, API5).
• Object IDs are sequential integers → BOLA enumeration is feasible (API1).
• Version string `v1` leaked by the health route → supports version enumeration (API9).
• No API documentation endpoint found → inventory must be maintained manually.

## 4. Artifacts & Reproduction

The following artifacts accompany this report:

- `deliverables/01_OWASP_API_Top10_Summary.md` — full OWASP summary.
- `deliverables/02_API_Endpoint_Map.md` — full endpoint map.
- `deliverables/endpoint_scan_results.json` — raw scan evidence.
- `scripts/recon.py` — reusable endpoint-discovery script.
- `target_app/app.py` — local practice Flask target.

### Reproduce the reconnaissance:

```
# 1. start the practice target
cd target_app && python3 app.py  # serves http://127.0.0.1:5000

# 2. in another terminal, run recon
cd scripts && python3 recon.py
```

Deliverables for Week 2 are complete: the OWASP API Security Top 10 summary and the API endpoint map, both backed by live reconnaissance evidence. The next phase (Week 3+) will use this map to test the prioritised authorization risks (API1/API5) against the authorized target.